ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Проект: FlightFinder - Веб-сервис для поиска авиабилетов

1. Общие сведения

Название проекта	FlightFinder
Тип проекта	Веб-приложение (Web-сервис)
Основные технологии	Python 3.12, Flask, SQLAlchemy

2. Цель и назначение проекта

Цель: Разработать веб-сервис для поиска и сравнения авиабилетов с возможностью фильтрации по различным параметрам, аналогичный Aviasales, но с уникальным дизайном и дополнительным функционалом.

Задачи, решаемые проектом:

- Предоставление пользователям актуальной информации о рейсах (на основе тестовых данных)
- Удобный поиск с фильтрацией по цене, наличию багажа, количеству пересадок
- Сохранение понравившихся рейсов в избранное
- Личный кабинет с возможностью загрузки аватара
- История поисковых запросов для авторизованных пользователей

3. Функциональные требования

3.1. Обязательный функционал

1. **Главная страница**

- Форма поиска с полями: откуда, куда, дата туда, дата обратно, количество пассажиров
- Блок популярных направлений (динамически генерируется)
- Информация о преимуществах сервиса
- Навигационная панель с ссылками на основные разделы

2. **Поиск и фильтрация**

- Поиск рейсов по направлениям и датам
- Фильтрация результатов:

  - По цене (диапазон)
  - По авиакомпаниям
  - По количеству пересадок
  - Только с багажом
- Сортировка результатов:

  - По возрастанию цены
  - По убыванию цены
  - По времени вылета
- Пагинация результатов (10 рейсов на странице)

3. **Карточка рейса**

- Отображение информации:

  - Город вылета и прилета (с кодами аэропортов)
  - Время вылета и прилета
  - Длительность перелета
  - Авиакомпания
  - Номер рейса
  - Наличие багажа
  - Количество пересадок
  - Цена
- Возможность добавить в избранное (для авторизованных)

4. **Регистрация и авторизация**

- Форма регистрации (имя пользователя, email, пароль)
- Форма входа
- Валидация данных
- Защита от повторной регистрации с теми же данными

5. **Личный кабинет**

- Просмотр профиля
- Загрузка и изменение аватара
- Список избранных рейсов

- История поисковых запросов

6. **REST API**

- Эндпоинт для автоподстановки городов (/api/airports)
- Эндпоинт для автоподстановки направлений (/api/destinations)

3.2. Дополнительный функционал

- **Избранное** — сохранение понравившихся рейсов в отдельный список
- **История поиска** — сохранение последних запросов пользователя
- **Анимации** — плавные переходы при наведении на карточки рейсов
- **Адаптивный дизайн** — корректное отображение на мобильных устройствах

4. Нефункциональные требования

4.1. Технологический стек

Компонент	Технология	Обоснование
Язык программирования	Python 3.12	Изучен в рамках курса
Веб-фреймворк	Flask 3.0.3	Легковесный, гибкий, изучен на занятиях
ORM	Flask-SQLAlchemy 3.1.1	Для работы с БД (требование проекта)
Формы	Flask-WTF 1.2.1	Валидация и защита форм
Аутентификация	Flask-Login 0.6.3	Управление сессиями пользователей
REST API	Flask-RESTful 0.3.10	Создание API (требование проекта)
Работа с изображениями	Pillow 10.2.0	Обработка аватарок
База данных	SQLite	Хранение данных (требование проекта)

4.2. Требования к коду

1. **Чистота кода:**
   - Соблюдение PEP 8
   - Осмысленные имена переменных и функций
   - Отсутствие копипаста (DRY принцип)
   - Комментарии к сложным участкам кода
2. **Качество проектирования:**
   - Модульная структура (разделение на blueprint'ы)
   - Использование классов (модели, формы)
   - Паттерн Factory для создания приложения
   - Разделение логики и представления
3. **Безопасность:**
   - Хеширование паролей (werkzeug.security)
   - Защита от CSRF (Flask-WTF)
   - Валидация загружаемых файлов
   - Ограничение размера загружаемых файлов

4.3. Требования к интерфейсу

- **Цветовая схема:** светлая тема с акцентами синего цвета (#4361ee)
- **Шрифты:** Inter, -apple-system, sans-serif
- **Адаптивность:** корректное отображение на всех устройствах
- **Интуитивность:** понятный интерфейс, не требующий обучения

5. Структура базы данных

5.1. Модели данных

```sql
-- Таблица пользователей
CREATE TABLE users (
    id INTEGER PRIMARY KEY,
    username VARCHAR(80) UNIQUE NOT NULL,
    email VARCHAR(120) UNIQUE NOT NULL,
    password_hash VARCHAR(256) NOT NULL,
    avatar VARCHAR(256) DEFAULT 'default_avatar.png',
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- Таблица рейсов
CREATE TABLE flights (
    id INTEGER PRIMARY KEY,
    origin VARCHAR(100) NOT NULL,
    destination VARCHAR(100) NOT NULL,
    origin_code VARCHAR(3) NOT NULL,
    destination_code VARCHAR(3) NOT NULL,
    departure_time DATETIME NOT NULL,
    arrival_time DATETIME NOT NULL,
    price INTEGER NOT NULL,
    airline VARCHAR(100) NOT NULL,
    baggage BOOLEAN DEFAULT 0,
    transfers INTEGER DEFAULT 0,
    flight_number VARCHAR(20) UNIQUE,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

```sql
-- Таблица избранного
CREATE TABLE favorites (
    id INTEGER PRIMARY KEY,
    user_id INTEGER NOT NULL,
    flight_id INTEGER NOT NULL,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id),
    FOREIGN KEY (flight_id) REFERENCES flights(id),
    UNIQUE(user_id, flight_id)
);

-- Таблица истории поиска
CREATE TABLE search_history (
    id INTEGER PRIMARY KEY,
    user_id INTEGER NOT NULL,
    origin VARCHAR(100) NOT NULL,
    destination VARCHAR(100) NOT NULL,
    date_from DATE,
    date_to DATE,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id)
);
```

6. Структура проекта

6.1. Корневая директория

main.py - главный файл приложения

- Содержит весь код проекта (конфигурация, модели, формы, маршруты)
- Точка входа для запуска приложения
- При первом запуске создает БД и генерирует 200 тестовых рейсов

requirements.txt - список зависимостей проекта

- Flask, Flask-SQLAlchemy, Flask-WTF, Flask-Login, Flask-RESTful, Pillow

.gitignore - исключает из Git: виртуальное окружение, БД, загруженные файлы, кеш Python

6.2. Папки в корне

- Содержит flightfinder.db (SQLite)

templates/ - папка с HTML шаблонами

- base.html - базовый шаблон (навигация, футер, flash-сообщения)
- index.html - главная страница (поиск, популярные направления)
- profile.html - личный кабинет (аватар, информация)
- favorites.html - избранное (сохраненные рейсы)
- auth/login.html - страница входа
- auth/register.html - страница регистрации
- flights/search.html - страница поиска
- flights/results.html - результаты поиска (фильтры, сортировка, пагинация)
- flights/detail.html - детальная информация о рейсе

static/ - папка со статическими файлами

- uploads/ - загруженные аватарки пользователей
- uploads/default_avatar.png - аватар по умолчанию

7. Тестовые данные

Для демонстрации работы сервиса создается 200 тестовых рейсов со следующими характеристиками:

- **Города:** 36 городов (Москва, Санкт-Петербург, Париж, Лондон, Нью-Йорк и др.)
- **Авиакомпании:** 12 компаний (Аэрофлот, Emirates, Turkish Airlines и др.)
- **Цены:** от 3 000 до 100 000 рублей

- **Пересадки:** 60% рейсов без пересадок, 30% с 1 пересадкой, 10% с 2 пересадками
- **Даты:** на 60 дней вперед от текущей даты

8. Возможности для развития

1. Подключение реального API авиакомпаний
2. Добавление онлайн-оплаты
3. Система отзывов о рейсах
4. Мобильное приложение
5. Уведомления о снижении цен
6. Интеграция с картами (показать на карте)
7. Система рекомендаций на основе истории поиска

9. Список использованных источников

- Документация Flask: https://flask.palletsprojects.com/
- Документация SQLAlchemy: https://www.sqlalchemy.org/
- Материалы курса Яндекс.Лицея

10. Заключение

Проект FlightFinder представляет собой полноценное веб-приложение, реализующее все требования учебной программы. В проекте использованы все изученные технологии, применены современные подходы к разработке, обеспечена безопасность и удобство использования.

Согласовано:

Разработчик: Вардумян Эрик Араратович

Дата: «15» марта 2026г.

Преподаватель: Грачёв Николай Романович